Weld Agent Study — Research Documentation & Paper-Drafting Reference

For: Dr. Jie Wu and Andrew Sadler **Status:** Working reference, v1.0 — consolidates the study design, the complete data inventory with rationale, the task set, the analysis plan, and scaffolding for a paper draft. **One-line summary:** A controlled study of how beneficial it is to consolidate task, communication, and code context in one workspace and make it accessible to an AI agent — operationalized as a context ladder (chat-only → +board → +GitHub) and measured by agent effectiveness and by user speed/effort/trust against each participant's own manual baseline, inside an integrated developer workspace called Weld.

---

1. Motivation & Research Questions

Software development is fragmented across many tools (issue trackers, chat, code hosts), and AI coding assistants are increasingly embedded in that workflow. A recurring open question in the literature is how much *context* an assistant needs to be useful: prior work shows repository-level context consistently improves an agent's ability to infer developer intent, and that developers' *perceptions* of AI assistance often diverge from their measured behavior. Weld — a workspace integrating task management, team chat, and (mocked) GitHub in one application — lets us manipulate exactly how much of a shared workspace an AI agent can see and act on, and measure the downstream effect on the user.

RQ1 — The value of consolidating context in one workspace. How beneficial is it to have task, communication, and code context unified and easily accessible *to an AI agent* in a single workspace, rather than fragmented or partially available? This is the "all-in-one" question: does consolidation produce a real, measurable benefit — to the agent (it can do more, more accurately) and, through it, to the user (faster, lower effort)?

RQ2 — The effect of agent context breadth (the test of RQ1). Does giving the agent progressively more of that unified context (chat → +board → +GitHub) make it more effective and speed the user up, relative to each user's own manual baseline? Where does added context help, and where does it not?

> The two RQs are one argument at two levels: RQ1 is the claim (unified context is beneficial), RQ2 is its controlled test (the context ladder). The C1→C2→C3 ladder *is* the integration measure — a C1 agent has fragmented/partial access, a C3 agent has fully unified access — so moving up the ladder literally is increasing integration.

The manual C0 baseline anchors the "benefit to the user" half: each agent condition is measured as improvement over the user working alone.

Hypotheses.

- **H1.** Broader-context conditions yield larger improvements over the user's manual baseline (faster task completion).
- **H2.** The benefit of context is *task-dependent* — minimal on comprehension and single-surface tasks, substantial on cross-surface tasks spanning chat → board → GitHub.
- **H3.** Broader context reduces self-reported cognitive load relative to baseline more than narrow context.
- **H4 (perception–behavior).** User trust and perceived speedup may not track the agent's objective effectiveness — users may trust an agent that *does more* even when it isn't faster or is occasionally wrong.

2. Study Design

Hybrid: within-subjects baseline + between-subjects agent condition.

- Every participant first completes **2 manual baseline tasks (C0)** with no agent — establishing their own manual speed and cognitive load.
- Each participant is then **randomly assigned to one agent condition** and completes **6 agent tasks** in that fixed condition.

The three agent context conditions (the independent variable):

Condition	**Agent can SEE**	**Agent can DO**
C1 — Chat-only	Chat only	Advise only; user executes
C2 — Chat + Board	Chat + board	Advise + act on board/chat
C3 — Full context	Chat + board + GitHub	Act across chat, board, and (mocked) GitHub

Design rationale (for the methods section):

- *Why hybrid.* The per-person C0 baseline lets each participant's agent-task performance be analyzed as a *delta from their own manual speed*, absorbing individual skill differences that would otherwise sit between the between-subjects conditions and sap power. Two baseline tasks (not one) stabilize the per-person anchor.

- *Why between-subjects on condition.* Holding the agent's access fixed per person avoids within-person contamination (once you've seen the full-context agent solve a task, you can't un-see how it should go) and keeps the session within a self-serve ~45-minute budget.
- *Why capability scales with context.* An agent cannot act on a board it cannot see, so visibility and capability are intentionally coupled. This bundling *is* the phenomenon under study ("does giving the agent more access help the user"); it is stated as a design decision, not treated as a confound.
- *Apparatus.* All participants use the same fictional seed project ("PetTracker"); only the agent's context differs. GitHub is mocked/seeded inside the app (no live API, no repo, write controls hidden), so participation is fully self-serve and nothing a participant does corrupts another's environment. The agent runs on a pinned OpenAI model (`gpt-4o-mini-2024-07-18`) at fixed temperature (0.2) with versioned prompts, so the model is a controlled, reproducible variable. (`gpt-4.1-mini` is held in reserve as a documented backup if the primary proves insufficient on C2/C3 tool-calling during the pilot; whichever model the pilot settles on is pinned for the entire study.)

3. Participants

- **Target N:** ~20–30 adults, 6 agent-task observations per participant plus baselines provide additional power.
- **Population:** adults 18+, ≥1 year programming experience, basic familiarity with issue trackers, team chat, and GitHub. Student-focused recruitment, not student-limited.
- **Recruitment:** self-serve link via course lists, departmental channels, developer communities.

4. The Task Set

Eight tasks, ~4–5 minutes each (~40–45 min + survey). All on the PetTracker seed. The agent tasks span three use-case types so we can locate *where* context breadth matters; comprehension and single-surface tasks serve as controls demonstrating that broader context does not *universally* help — which is what makes the cross-surface result credible.

Manual baseline (C0, everyone, no agent):

- **B1** — comprehension + single action (read chat, find and move a named card).
- **B2** — cross-surface (read a chat request, check a card's board status, post a status reply).

Agent tasks (assigned condition):

Task	Type	Context dependency	What it tests
T1 — "What needs doing?"	Comprehension/advice	Low	Chat-only competitive; do C2/C3 finish faster by also acting?
T2 — "Catch me up"	Comprehension/advice	Low–Med	Do richer summaries (board/GitHub) reduce follow-up work?
T3 — Draft a status message	Single-surface (chat)	Low (control)	Extra context shouldn't help — validates the design
T4 — Set priorities on the board	Single-surface (board)	Medium	C1 (advise-only) vs. C2/C3 (act): cost of advising vs. doing
T5 — Take a bug from chat to the board	Cross-surface	High	C2/C3 resolve chat→card and act; C1 can't — core H1/H2
T6 — Link the PR to the card + note it in chat	Cross-surface	Highest	Only C3 has GitHub to do it end-to-end — justifies the top rung

Key design point for the paper: T6 is constructed so the correct pull request is identifiable *only* with GitHub context (chat refers to it vaguely; the PR's title/body disambiguates). If C3 does not beat C2 on T6, the GitHub context is not pulling its weight — and the data will show that. This is the cleanest single test of the top of the context ladder.

5. Complete Data Inventory (what we collect and why)

Everything joins to one anonymized `participant_id`. Four categories.

Category A — Behavioral telemetry (auto-logged during the session)

Data	**Why we collect it**
Session metadata: start/end, `agent_condition`, phase transitions, abandonment point	Frame each session; enable per-condition comparison; handle dropouts
Per-task timing: shown/completed/skipped timestamps, instruction re-opens	Core productivity measure; re-opens proxy confusion
Board events: create/open/edit/move/assign/label/priority/search	Board actions by user vs. agent; RQ1 supporting evidence (division of labor)
Chat events: message sent (length, has-mention, has-link), edits, channel switches, thread opens, mention/link clicks	Communication & navigation; RQ1 supporting evidence
GitHub-view events: view opened, issue/PR opened, card↔issue/PR links	Whether/how GitHub context is used (esp. C3)
Navigation: view changes, searches	View-switch rate (context-switching measure)
Cross-reference events (card-from-chat, chat-from-card, PR-from-card, card-from-PR)	**Headline integration-usage metric**; how context is traversed
Agent events: invocation, `agent_context_sent` (snapshot of exactly what the agent received), response (latency, type, # tool calls, model, temp), each tool call (name, allowed?, executed?), tool-rejected anomalies	Agent effectiveness (RQ2); **`agent_context_sent` is the proof that scoping held** — without it the context manipulation rests on trust, not evidence
User-to-agent reactions: suggestion accepted/rejected, action undone, output edited	How unified context shifts work from user to agent; RQ1 supporting evidence + over/under-reliance

Category B — Survey responses (self-report, once at the end)

Instrument	**Why**
NASA-TLX (6 subscales)	Cognitive load (H3); validated, comparable to literature

Instrument	Why
System Usability Scale (10 items)	Overall usability; comparable to published norms; `sus_05` ("well integrated") is research-relevant
Integration items (5)	Perceived ease of working across surfaces
Trust-in-agent block (6 + 2 items)	Agent trust, reliability, reliance willingness; `trust_context` should rise C1→C2→C3; `trust_speed_perceived` vs. actual time = perception–behavior gap (H4)
Open-ended (4 prompts)	Qualitative depth; most-useful/most-frustrating moments; thematic analysis
Demographics (8 bucketed fields)	Sample description; covariates (experience, prior tool use)
Optional follow-up consent	Recruit interview subset; stored separately

Category C — Derived metrics (computed from A at analysis time)

Metric	From	Serves
Time-to-first-action; per-task completion time	A timing	Productivity
Task completion rate	A + rubric (D)	Productivity
View-switch rate	A navigation	Context-switching
Cross-reference rate	A cross-ref vs. card-open	Integration usage
Baseline-normalized task time (agent task ÷ own C0)	A + baseline	**Core RQ2 comparison across C1/C2/C3**
Agent delegation ratio (agent- vs. user-performed actions)	A agent + action events	RQ1 (how much unified context offloads from the user)
Agent latency, tool-call counts	A agent events	Agent efficiency/cost

Metric	From	Serves
Perception–behavior gap (perceived speed/trust vs. normalized time)	A + B	**H4 — likely the most novel finding**

Category D — Manual-coded data (researcher-scored post-hoc)

Coded item	Why
Per-task correctness (0–3 rubric)	Speed is meaningless without correctness — a fast-but-wrong agent isn't effective
Who-did-it (agent / user / mixed)	RQ1 (division of labor as context unifies)
Agent error type (wrong target / hallucination / couldn't-due-to-scoping / tool error)	Diagnose *why* a condition underperforms; "couldn't due to scoping" is signal, not noise
Thematic coding of open-ended responses	Qualitative findings; two coders, report Cohen's kappa on a sample

What we deliberately do NOT collect (privacy + IRB)

No keystroke logging (final submitted text only), no mouse-movement/hover tracking, no screen or camera recording, no data from other browser tabs, no data about any project other than the participant's PetTracker instance. This exclusion list is both an ethical commitment and what keeps the study in exempt/expedited IRB territory.

Data handling

Encrypted institutional storage; access limited to the research team; pseudonymous IDs throughout; follow-up contact emails (if any) stored separately, linked by token, deleted after the follow-up window; open-ended text screened for identifying content before any release.

6. Analysis Plan

- **Primary (RQ2):** mixed-effects model — `task_time ~ condition × task_type + (1 | participant)`, with C0 as the per-person anchor (analyze baseline-normalized times). Report effect sizes and confidence intervals, not just p-values.
- **H2 test:** the `condition × task_type` interaction — is the condition effect larger for cross-surface tasks? This is the key result.
- **Correctness:** always reported alongside time; a condition that is faster but less correct is not "better." Compare correctness distributions across conditions.
- **H3:** NASA-TLX deltas from baseline across conditions.

- **H4:** correlate `trust_speed_perceived` and trust items against actual normalized time and correctness; look for divergence. Correlate `trust_rely` vs. `trust_verify` for over/under-reliance.
- **RQ1 (integration value):** synthesize across conditions — does agent effectiveness (correctness, ability to complete cross-surface tasks, latency) and user benefit (baseline-normalized time, cognitive load) rise as the agent's context becomes more unified (C1→C2→C3)? The ladder read as a whole *is* the integration measure. Supporting evidence: delegation ratio and verification/repair patterns from telemetry (how the unified context changes what the agent can take off the user's plate) and thematic analysis of open-ended responses.
- **Scoping validity check:** before any analysis, verify from `agent_context_sent` logs that C1 never received board/GitHub data and C2 never received GitHub. Report this as a manipulation check.

7. Open Items for the Two of You to Decide

- Pre-registration on OSF (recommended; strengthens the COI story), is this something we need to do?
- IRB: Do we have to make an IRB application or not?
- Whether to run the optional follow-up interviews (script already drafted).
- Collect Email from users

Jun 3, 2026

1. Email (instruction)
2. Smoke tests
3. Find users